Universidad Nacional de Colombia
Sede Bogotá

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial

EnvíoFácil

Laboratorio: del sector profesional al código

Arquitectura, subarquitectura, SOLID
y patrones de diseño

Ronald Arturo Chávez Cabrera

Ingeniería de Software II (2016702)
Docente: Sergio Enrique Vargas Pedraza

Ruta A · Desarrollo
21 de octubre de 2026

1. Dominio y problema

Para este laboratorio escogí un problema sencillo de una tienda: organizar sus envíos. Si los pedidos se anotan en mensajes y planillas distintas, es fácil repetir datos, equivocarse al calcular el costo o perder el número de una guía. EnvíoFácil reúne esas tareas en un programa donde se ingresan los datos del paquete, se consulta el precio y se confirma el registro.

Actores. El encargado de la tienda utiliza el programa desde la consola. El destinatario aparece en los datos de entrega, pero no usa la aplicación. Para representar la comunicación con empresas de transporte, incluí dos proveedores simulados: RutaLocal y PaqueteYa. Al registrar un envío, el programa actualiza el historial y muestra un aviso.

Alcance. Desarrollé una versión de prueba que funciona en un solo proceso y guarda la información en memoria. Permite consultar los envíos registrados durante la ejecución. Las tarifas y las transportadoras son ficticias.

ID	Requisito funcional
RF01	Construir y validar una solicitud con origen, destino, destinatario y peso positivo.
RF02	Agregar valor declarado, indicación de fragilidad, instrucciones y referencia opcionales.
RF03	Cotizar con modalidad estándar o exprés, sin registrar el envío.
RF04	Registrar la solicitud mediante una transportadora seleccionada.
RF05	Generar una guía estándar o exprés y conservar el aviso de fragilidad.
RF06	Consultar un envío registrado mediante su identificador.
RF07	Actualizar el historial y emitir una notificación local después de guardar.
RF08	Auditar las operaciones exitosas de guardado y consulta sin copiar datos personales.

Atributo	Criterio del proyecto
Mantenibilidad	Agregar una tarifa u observador sin editar ServicioEnvios; comprobarlo con pruebas.
Fiabilidad	No guardar si falla la transportadora; un fallo de notificación no revierte el registro ni bloquea otros observadores.
Privacidad	Usar datos ficticios y limitar la bitácora a operación e ID. La autenticación queda fuera del prototipo.
Rendimiento	Cotización local menor a 100 ms con un operador. Es una meta de diseño; no se midió el rendimiento.

2. Arquitectura macro

Escogí un **monolito modular** porque el programa tiene un alcance pequeño y puede funcionar en un solo proceso de Python. Separé el código por responsabilidades: la consola recibe los datos, el componente Envíos realiza las operaciones y el módulo de composición conecta las clases que se van a utilizar.

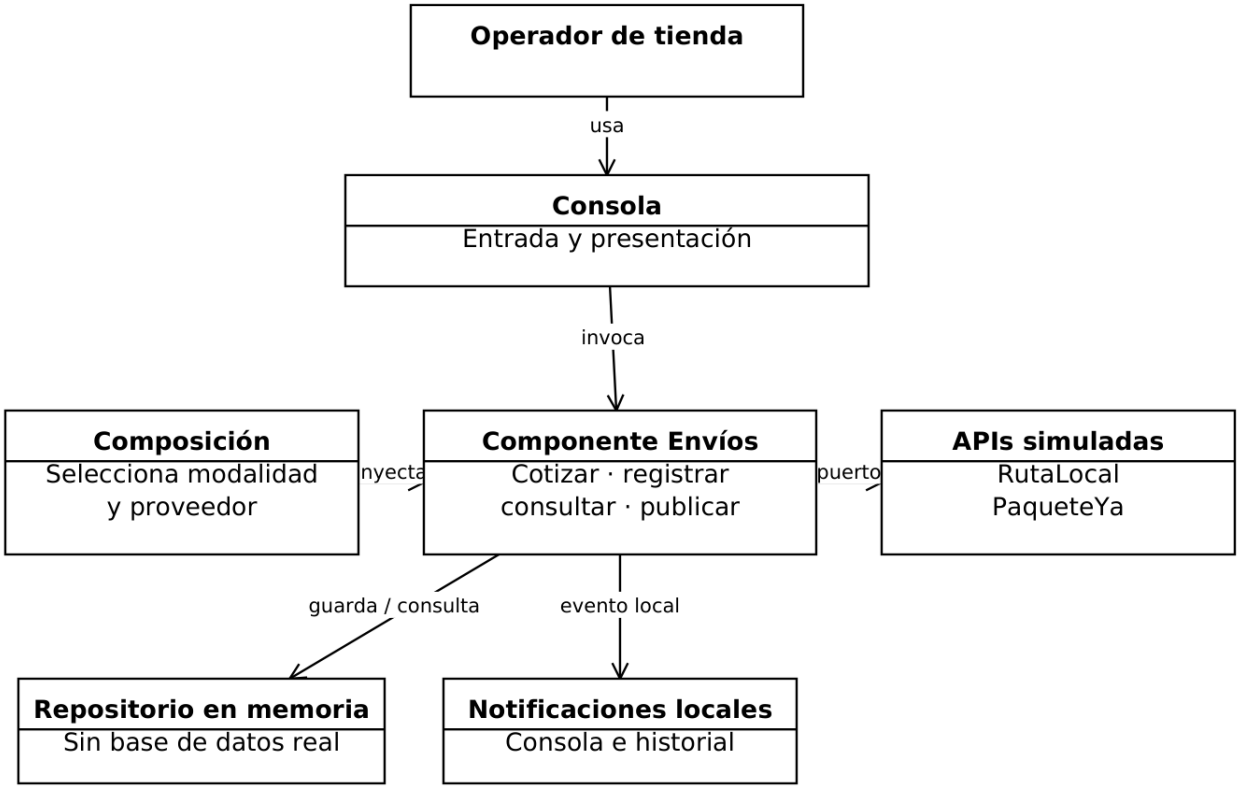


Figura 1. Módulos e integraciones del prototipo. Las APIs son simulaciones locales.

Atributo	Justificación frente a microservicios
Mantenibilidad	Las carpetas y puertos separan responsabilidades. Para un componente pequeño, distribuir procesos añadiría contratos de red y despliegues sin una necesidad actual.
Fiabilidad	El flujo se depura en un proceso. Los microservicios podrían aislar fallos, pero exigirían reintentos y coordinación entre servicios que este caso no necesita.
Privacidad	Mantengo los datos de prueba en memoria y reduzco las copias. Los microservicios agregarían comunicaciones y controles entre servicios.
Rendimiento	Las llamadas internas evitan el costo de la red. Si aparecen equipos independientes o cargas muy distintas, tendría sentido reconsiderar la división.

Para este trabajo, mantener todo en un proceso facilita la ejecución y la revisión del código. El almacenamiento en memoria se pierde al cerrar el programa; una versión para uso diario necesitaría guardar los registros de forma permanente.

3. Componente y subarquitectura

Trabajé sobre el componente **Gestión de envíos** porque allí se relacionan el cálculo del precio, la generación de la guía y la comunicación con la transportadora. Utilicé arquitectura **hexagonal** para separar esas reglas de la consola y de las clases que guardan o envían información.

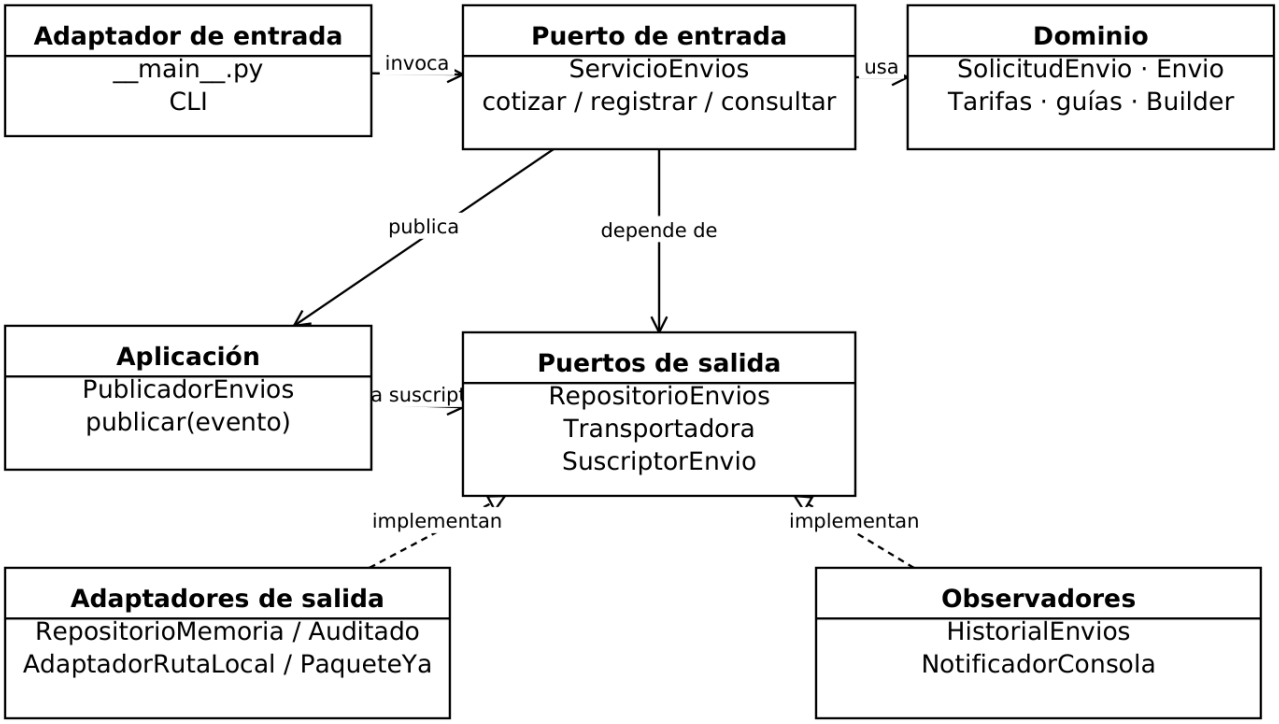


Figura 2. Dependencias hacia el dominio y los contratos. ABC indica una clase base abstracta.

Parte	Clases y módulos
Dominio	SolicitudEnvio, Envio, EnvioRegistrado, SolicitudBuilder, PoliticaTarifa, tarifas, Guia y creadores; carpeta dominio/.
Entrada y aplicación	ServicioEnvios expone cotizar, registrar y consultar. PublicadorEnvios distribuye eventos; carpeta aplicacion/.
Puertos de salida	RepositorioEnvios, Transportadora y SuscriptorEnvio; aplicacion/puertos.py.
Adaptadores	RepositorioMemoria, RepositorioAuditado, adaptadores de transportadoras y observadores; carpeta adaptadores/.
Composición y CLI	composicion.py elige objetos concretos; __main__.py transforma las entradas del usuario en llamadas al componente.

ServicioEnvios es la entrada al componente: desde allí se puede cotizar, registrar y consultar. Las reglas del dominio no dependen de la consola ni de una transportadora específica. Los objetos concretos se eligen en composicion.py y se entregan al servicio por su constructor.

4. Diseño de clases

En el siguiente diagrama presento las clases que intervienen en el registro y la consulta de un envío. Los diagramas de las fichas muestran con más detalle las clases de cada patrón, usando los mismos nombres que aparecen en el código.

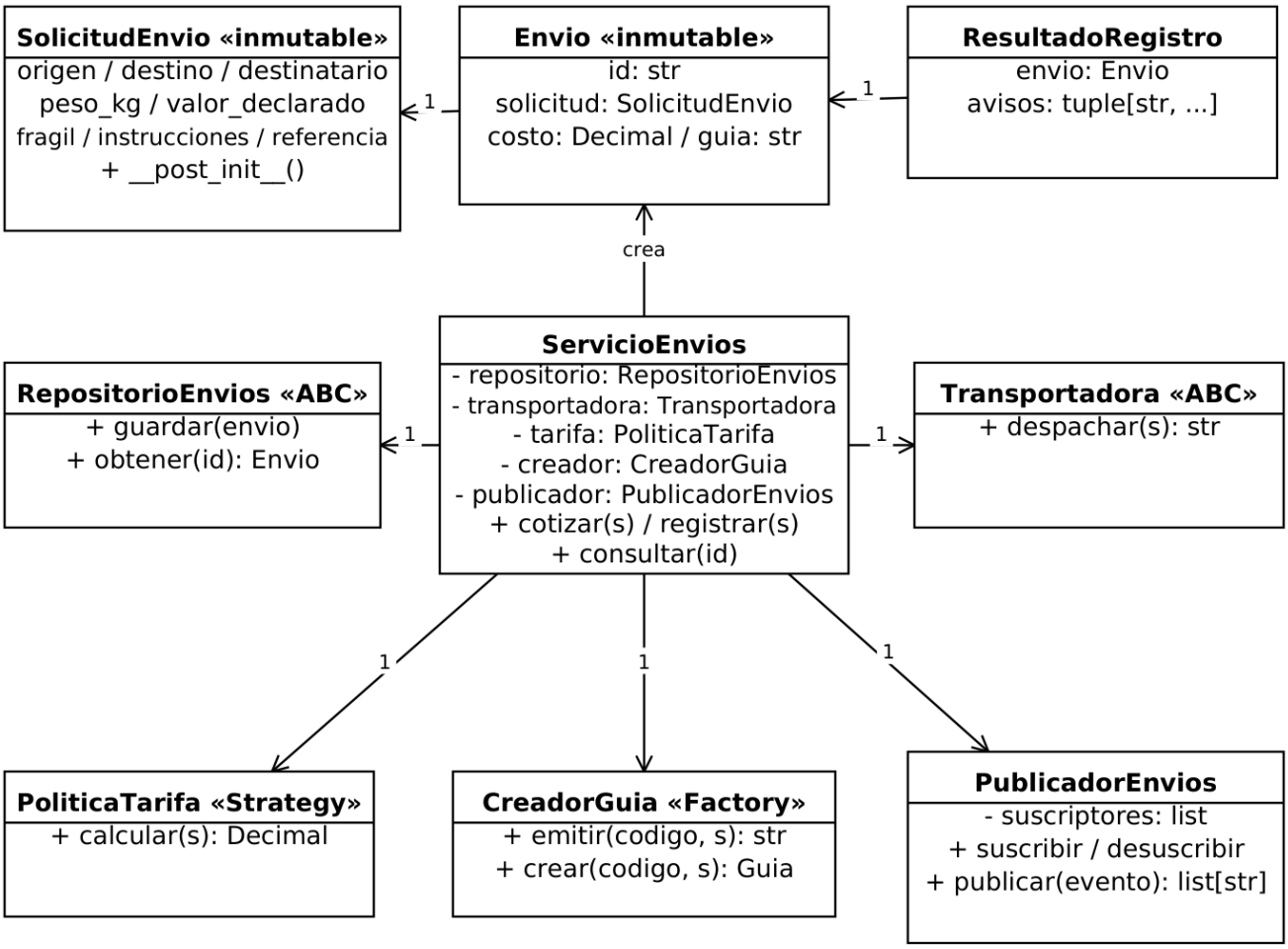


Figura 3. Vista central. Una flecha de uso apunta a la dependencia; “1” indica una instancia asociada.

Flujo del registro

El registro comienza con una SolicitudEnvio validada. ServicioEnvios calcula el costo, pide un código a la transportadora y genera la guía. Luego guarda el envío y publica el evento EnvioRegistrado. El resultado incluye el envío y cualquier aviso de error que haya ocurrido al notificar.

Reglas de las interfaces

La tarifa devuelve Decimal finito y no negativo. La transportadora devuelve un código no vacío o propaga un error. El repositorio rechaza un ID repetido con ValueError y una consulta inexistente con KeyError. SolicitudEnvio y Envio son inmutables; el decorador conserva los contratos del repositorio.

PublicadorEnvios mantiene entre cero y varios suscriptores. NotificadorConsola muestra un aviso e HistorialEnvios conserva los IDs registrados. Ninguno necesita conocer al otro.

5. Evidencias de SOLID

Principio	Evidencia concreta
S · Responsabilidad única	SolicitudEnvio valida los datos; las tarifas calculan; los adaptadores traducen las APIs; el repositorio almacena. ServicioEnvios coordina el caso de uso, sin implementar esos detalles.
O · Abierto/cerrado	PoliticaTarifa permite una TarifaConvenio nueva sin editar ServicioEnvios. test_ocp_nueva_estrategia_sin_modificar_servicio la define en el test y comprueba cotización y registro.
L · Sustitución de Liskov	RepositorioMemoria y RepositorioAuditado cumplen los mismos resultados y excepciones. test_liskov_repositorios_guardar_consultar_y_errores ejecuta el mismo contrato contra ambos. Otra prueba sustituye las transportadoras.
I · Segregación de interfaces	Transportadora solo exige despachar; SuscriptorEnvio solo actualizar. RepositorioEnvios reúne guardar y obtener porque el componente usa ambos. Los observadores no implementan funciones de transporte.
D · Inversión de dependencias	ServicioEnvios recibe los puertos, PoliticaTarifa y CreadorGuia por constructor. composicion.py crea las clases concretas y las inyecta; el caso de uso no instancia APIs ni repositorios.

Antes y después 1: cálculo de precios (O)

Una forma de resolver el precio sería colocar un condicional por cada modalidad dentro del servicio. El problema es que habría que editar ese método cada vez que apareciera otra tarifa. Comparé esa opción con la solución basada en Strategy. Los fragmentos son ejemplos de diseño, no versiones anteriores del repositorio.

```
ANTES
if modalidad == "estandar":
    costo = 7000 + 2000 * peso
elif modalidad == "express":
    costo = 12000 + 3000 * peso

DESPUÉS — ServicioEnvios.cotizar()
costo = self._tarifa.calcular(solicitud)
```

Por eso separé las fórmulas en TarifaEstandar y TarifaExpress. Para agregar otra tarifa, se crea una clase y se selecciona en composicion.py. ServicioEnvios sigue llamando al mismo método calcular().

Antes y después 2: persistencia y auditoría (S y D)

```
ANTES — responsabilidades mezcladas
self.envios[envio.id] = envio
self.bitacora.append("GUARDAR " + envio.id)

DESPUÉS — ServicioEnvios.registrar()
self._repositorio.guardar(envio)
```

RepositorioMemoria resuelve el almacenamiento. RepositorioAuditado lo envuelve y registra la operación después de completarla. Puedo quitar la auditoría cambiando la composición, sin alterar el caso de uso.

6. Patrones creacionales

6.1. Factory Method

Problema. La guía estándar y la exprés tienen diferencias. La segunda debe incluir un aviso de prioridad alta. Necesito que el servicio pueda emitir cualquiera de las dos sin decidir cómo se construye cada una.

Solución y alternativa. CreadorGuia.emitir() llama al método crear(), que cada creador implementa para producir su guía. También añade el aviso de fragilidad cuando corresponde. Otra opción sería usar un switch dentro del servicio, pero eso obligaría a modificarlo cada vez que se agregue un tipo de guía.

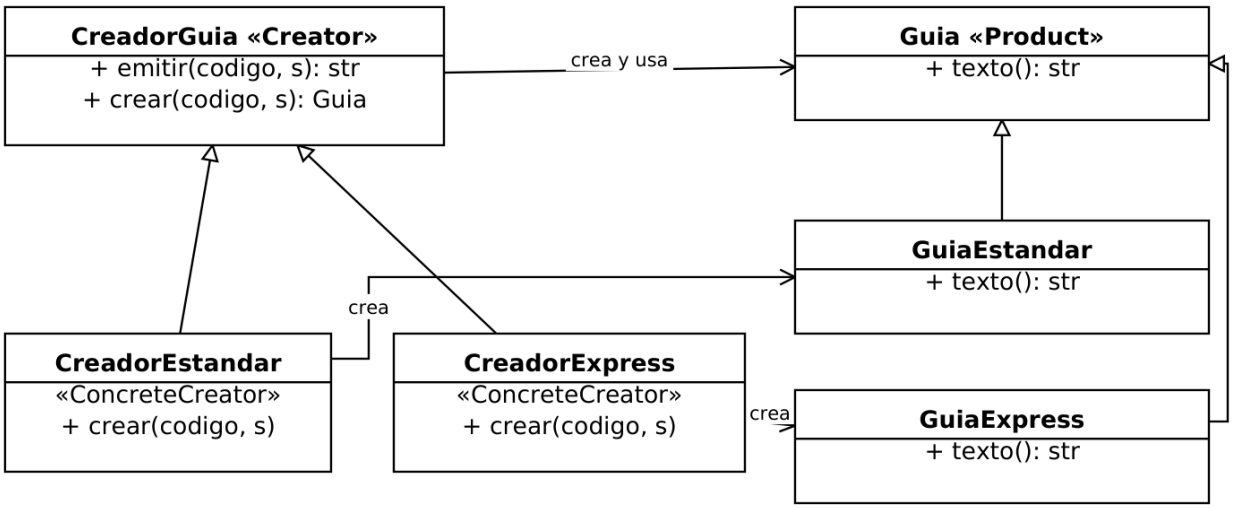


Figura 4. Creadores y productos de Factory Method.

Participantes. Creator = CreadorGuia; ConcreteCreators = CreadorEstandar y CreadorExpress; Product = Guia; ConcreteProducts = GuiaEstandar y GuiaExpress. **SOLID:** O y D. **Código:** dominio/guias.py.

6.2. Builder

Problema. Una solicitud necesita origen, destino, destinatario y peso. También puede llevar valor declarado, instrucciones, referencia e indicación de fragilidad. Pasar todo en una sola llamada posicional hace más fácil confundir los argumentos.

Solución y alternativa. Con SolicitudBuilder agrego los datos por pasos y al final obtengo una solicitud validada. Podría usar un constructor con argumentos nombrados, algo práctico en Python. Elegí Builder porque la consola recoge los datos uno por uno y varios son opcionales.

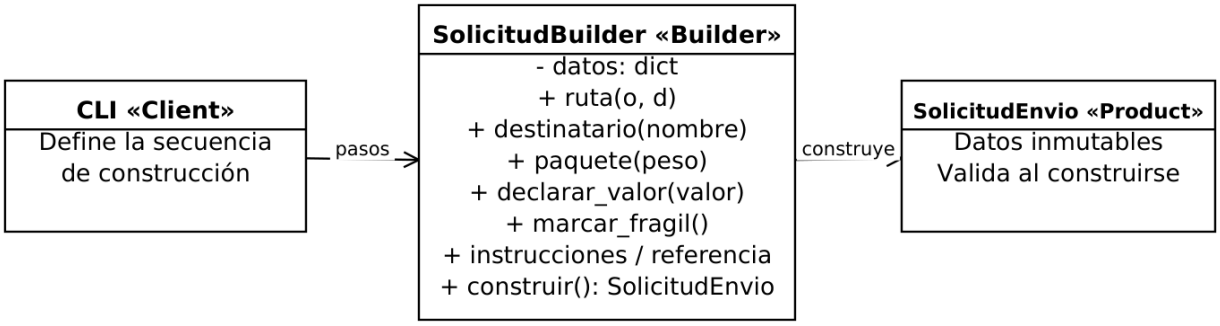


Figura 5. Builder fluido. La CLI dirige la secuencia; no requiere una clase Director.

Participantes. Builder concreto = SolicitudBuilder; Product = SolicitudEnvio; Client = CLI. **SOLID:** S separa el ensamblado de la coordinación del envío. **Código:** dominio/constructor.py y dominio/modelos.py.

7. Patrones estructurales

7.1. Adapter

Problema. RutaLocal recibe un diccionario con peso en gramos y devuelve tracking. PaqueteYa recibe argumentos separados, peso en kilos y devuelve un texto. El servicio necesita una única operación despachar(solicitud).

Solución y alternativa. Cada adaptador convierte los datos y normaliza el código de respuesta. Descarté llamar las APIs directamente desde ServicioEnvios porque mezclaría reglas de negocio con formatos de proveedores. Facade simplificaría llamadas, pero aquí el problema principal es la incompatibilidad de interfaces.

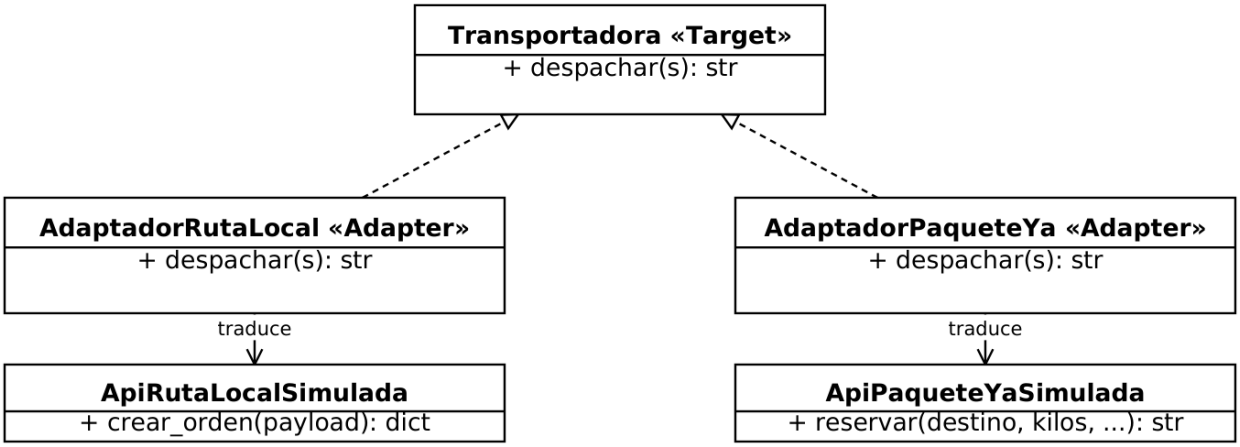


Figura 6. Los adaptadores implementan el puerto y traducen las APIs ficticias.

Participantes. Target = Transportadora; Adapters = AdaptadorRutaLocal y AdaptadorPaqueteYa; Adaptees = ApiRutaLocalSimulada y ApiPaqueteYaSimulada; Client = ServicioEnvios. **SOLID:** D y O. **Código:** adaptadores/transportadoras.py; contrato en aplicacion/puertos.py.

7.2. Decorator

Problema. Además de guardar y consultar envíos, necesito dejar un registro de esas operaciones. Quiero poder agregar o retirar esa auditoría sin cambiar la clase que almacena los datos.

Solución y alternativa. RepositorioAuditado recibe otro repositorio y usa sus métodos. Cuando una operación termina bien, la anota en la bitácora. Consideré heredar de RepositorioMemoria, pero así la auditoría quedaría ligada a ese tipo de almacenamiento. Con el decorador puedo reutilizarla con otro repositorio que cumpla el mismo contrato.

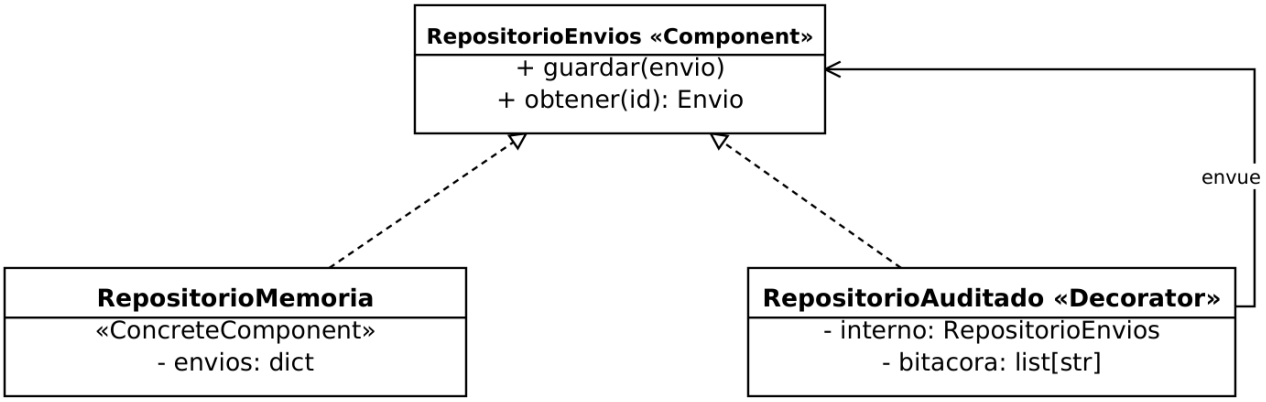


Figura 7. Decorator conserva la interfaz y envuelve otra implementación del puerto.

Participantes. Component = RepositorioEnvios; ConcreteComponent = RepositorioMemoria; Decorator concreto = RepositorioAuditado; objeto envuelto = interno. **SOLID:** S, O y L. **Código:** adaptadores/repositorios.py. Registro la operación solo después de que la delegación termine correctamente.

8. Patrones de comportamiento

8.1. Strategy

Problema. La cotización estándar y la exprés utilizan fórmulas distintas, pero ambas deben poder usarse desde el mismo caso de uso. Una tarifa futura no debería introducir otro condicional en ServicioEnvios.

Solución y alternativa. ServicioEnvios recibe una PoliticaTarifa y le pide calcular el precio. Así evito un condicional por cada modalidad dentro del servicio. State se usaría para cambiar el comportamiento según el estado del envío; aquí lo que cambia es la fórmula elegida para cobrar.

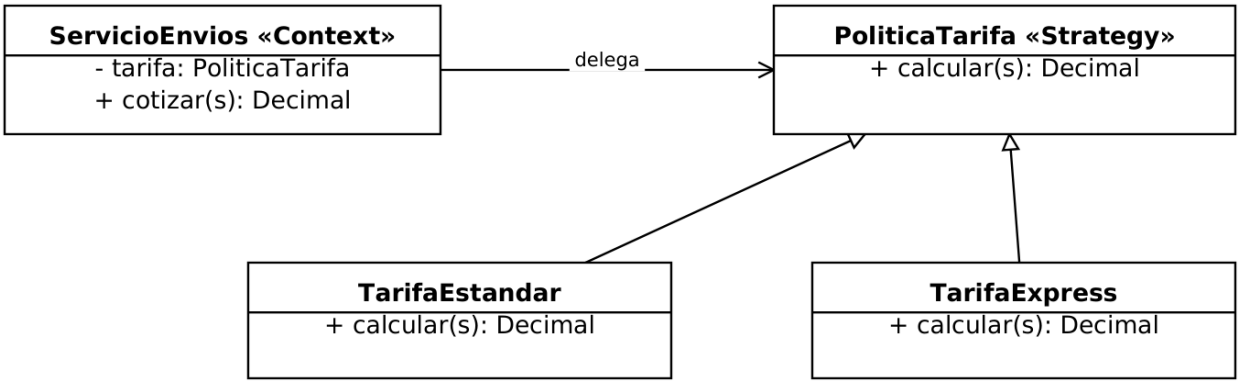


Figura 8. El contexto delega el algoritmo en una estrategia intercambiable.

Participantes. Context = ServicioEnvios; Strategy = PoliticaTarifa; ConcreteStrategies = TarifaEstandar y TarifaExpress. **SOLID:** O y D. **Código:** dominio/tarifas.py y aplicacion/servicio.py. Para 2 kg y COP 100.000 declarados: estándar = 7.000 + 2.000 × 2 + 1.000 = 12.000; exprés = 12.000 + 3.000 × 2 + 1.000 = 19.000.

8.2. Observer

Problema. Después de registrar un envío, el historial y la consola deben reaccionar sin que el servicio invoque directamente a cada implementación. Otro canal podría agregarse más adelante.

Solución y alternativa. PublicadorEnvios recibe el evento y avisa a los objetos suscritos. Podría llamar directamente al historial y a la consola, pero cada nuevo canal obligaría a editar esas llamadas. Para este programa es suficiente que los avisos se procesen dentro de la misma ejecución, sin un servicio externo de mensajería.

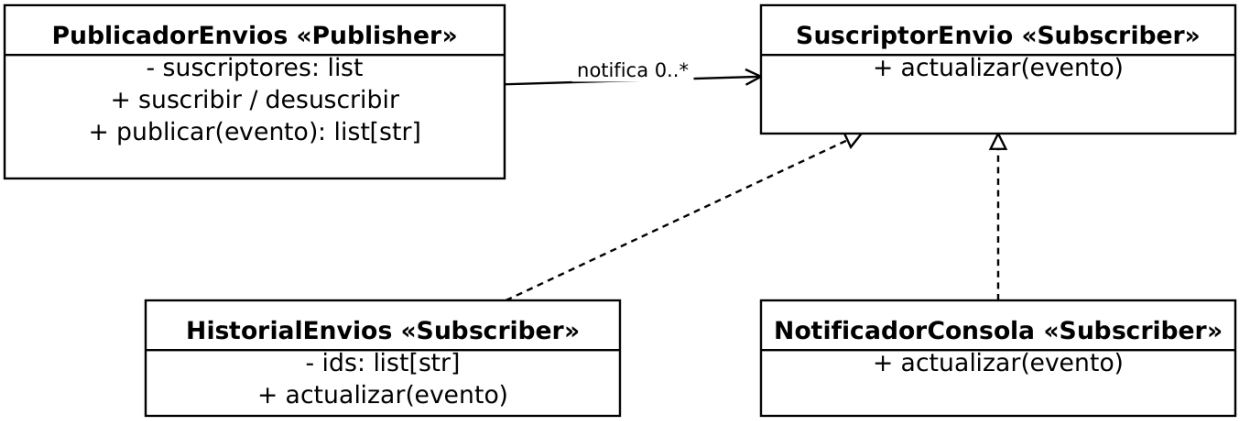


Figura 9. Una publicación recorre los observadores suscritos.

Participantes. Publisher = PublicadorEnvios; Subscriber = SuscriptorEnvio; ConcreteSubscribers = HistorialEnvios y NotificadorConsola; evento = EnvioRegistrado. **SOLID:** O, S e I. **Código:** aplicacion/eventos.py, aplicacion/puertos.py y adaptadores/notificaciones.py. Los errores se devuelven como avisos y no impiden ejecutar otros observadores.

9. Implementación y evidencias

Realicé la Ruta A con Python. El programa y las pruebas usan la biblioteca estándar, por lo que no hace falta instalar paquetes adicionales. Ejecuté la demo y las pruebas con **Python 3.12.14**. En las carpetas del proyecto están el código, el README, los diagramas y los resultados de ejecución.

Repositorio del proyecto:
https://github.com/RonaldChavez5/enviofacil-laboratorio

Cómo ejecutar el programa

```
python -m enviofacil demo
python -m enviofacil interactivo
python -m unittest discover -s tests -v
```

Los comandos se ejecutan en la carpeta que contiene README.md y enviofacil. En Windows también puede usarse py; en Linux o macOS, python3.

Pruebas	Cantidad	Qué comprobé
Dominio	8	Datos obligatorios, opciones, valores inválidos, inmutabilidad, fórmulas y productos de guía.
Contratos	5	Sustitución de repositorios y transportadoras, errores, auditoría y traducción del payload.
Casos de uso	6	Registro, consulta, cotización sin efectos, extensión OCP, tarifa inválida y fallos del proveedor.
Observer	3	Varios suscriptores, ausencia de duplicados, desuscripción y aislamiento de errores.
Total	22	Todas las pruebas terminaron correctamente.

Resultado de las pruebas

```
----------------------------------------------------------------------
Ran 22 tests in 0.001s

OK
```

Evidencia completa: evidencias/pruebas.txt. La prueba de OCP añade TarifaConvenio dentro del test, inyecta esa estrategia en el mismo ServicioEnvios y verifica una cotización y un registro de COP 9.000, sin editar el contexto.

Resultado de la demo y del modo interactivo

Ejecución	Resultado observado
Estándar + RutaLocal	COP 12.000; guía RL-0001; consulta coincide; 1 registro en historial; 2 operaciones auditadas; sin avisos de error.
Exprés + PaqueteYa	COP 19.000; guía PY-0001 y prioridad alta; consulta coincide; 1 registro en historial; 2 operaciones auditadas; sin avisos de error.
Consola interactiva	Ingreso de datos, cotización y confirmación con datos ficticios. El registro y su consulta finalizaron correctamente.

Archivos: evidencias/demo.txt y evidencias/interactivo.txt guardan las salidas completas. Cada ejecución genera IDs nuevos. Las pruebas comprueban las funciones del programa; no hice pruebas de carga ni conexiones con transportadoras reales.

10. Tabla de trazabilidad

Organicé los ocho requisitos en esta tabla para indicar dónde se resuelve cada uno, qué clases participan y cómo se relacionan con los patrones y con SOLID. Todos corresponden al componente Gestión de envíos.

RF / parte	Clases principales	Patrón	SOLID / evidencia
RF01 Dominio y CLI	SolicitudBuilder SolicitudEnvio	Builder	S test_builder_rechaza_datos_incompletos; test_valida_peso_y_valor
RF02 Dominio y CLI	SolicitudBuilder SolicitudEnvio	Builder	S test_builder_campos_opcionales; test_producto_inmutable_e_independiente_del_builder
RF03 Aplicación y dominio	ServicioEnvios PoliticaTarifa TarifaEstandar / Express	Strategy	O, D test_tarifas_con_valores_esperados; test_cotizar_no_registra_ni_publica
RF04 Aplicación y salida	ServicioEnvios Transportadora AdaptadorRutaLocal / PaqueteYa	Adapter	D, L test_adapter_traduce_payload_y_respuesta; test_liskov_transportadoras_intercambiables
RF05 Dominio	CreadorGuia CreadorEstandar / Express GuiaEstandar / Express	Factory Method	O, D test_factory_method_productos_distintos; test_factory_conserva_aviso_fragil
RF06 Aplicación y salida	ServicioEnvios RepositorioEnvios RepositorioMemoria / Auditado	Decorator	L, D test_liskov_repositorios_guardar_consultar_y_errores
RF07 Aplicación y salida	PublicadorEnvios SuscriptorEnvio HistorialEnvios NotificadorConsola	Observer	O, I test_varios_suscriptores_sin_duplicados; test_error_observador_no_revierte_registro_ni_bloquea_otros
RF08 Salida	RepositorioAuditado RepositorioMemoria	Decorator	S, L test_decorator_audita_solo_operaciones_exitosas

Relación con las pruebas

Para RF03, la prueba de OCP comprueba que una tarifa nueva funciona con el mismo servicio. Para RF04 y RF07, las pruebas revisan qué ocurre cuando falla la transportadora o un observador. También comprobé que los repositorios respondan igual ante consultas inexistentes e IDs repetidos.

Los nombres completos de las pruebas aparecen en tests/test_envios.py. La tabla de README.md relaciona cada patrón con su archivo y clase para ubicarlo directamente en el proyecto.

11. Prueba de evolución

Para revisar si el diseño permite hacer cambios, planteé tres solicitudes que podría tener una tienda. En cada caso indico qué habría que agregar, qué archivos habría que modificar y qué clases podrían conservarse.

Escenario 1. Nueva transportadora “MensajeríaCentro”

Creadas	Modificadas	Sin cambios
AdaptadorMensajeriaCentro y el cliente de su API. Un test de traducción y contrato.	composicion.py: registrar y seleccionar el adaptador. __main__.py: actualizar las opciones visibles del proveedor.	ServicioEnvios, SolicitudEnvio, tarifas, creadores de guía, repositorios y publicador.

El nuevo adaptador implementaría despachar(solicitud) y devolvería el código de la guía. La conversión al formato de la transportadora quedaría dentro de esa clase. Las pruebas con RutaLocal y PaqueteYa muestran cómo se puede sustituir un proveedor. MensajeríaCentro es una propuesta para este escenario.

Escenario 2. Tarifa de convenio de COP 9.000

Creadas	Modificadas	Sin cambios
TarifaConvenio, implementación de PoliticaTarifa.	composicion.py: inyectar la tarifa cuando corresponda al convenio. Si la opción se expone al usuario, actualizar la CLI.	ServicioEnvios, repositorios, transportadoras, publicador y guías. Puede reutilizarse la guía estándar.

Comprobé este caso en test_ocp_nueva_estrategia_sin_modificar_servicio. La prueba define TarifaConvenio, la entrega al servicio y verifica que tanto la cotización como el registro usen COP 9.000. No fue necesario cambiar ServicioEnvios.

Escenario 3. Aviso adicional por SMS

Creadas	Modificadas	Sin cambios
NotificadorSms, implementación de SuscriptorEnvio, y su cliente de mensajería.	composicion.py: suscribir el canal. SolicitudEnvio, SolicitudBuilder y __main__.py: incorporar teléfono y autorización de contacto.	ServicioEnvios, PublicadorEnvios, HistorialEnvios, NotificadorConsola, tarifas, guías y repositorios.

El nuevo observador recibiría EnvioRegistrado. Las pruebas de múltiples suscriptores y aislamiento de errores comprueban el mecanismo local. El SMS real requiere teléfono y autorización de contacto: por eso también identifico cambios en el objeto de solicitud, su constructor y la captura. El observador enviaría el mensaje solo si la autorización lo permite; no existe un canal SMS real en esta versión.

Límite del principio abierto/cerrado

Los cambios anteriores aprovechan las interfaces que ya existen. Si un requisito pide datos nuevos, como el teléfono del destinatario, también habrá que modificar la solicitud y su captura. Aplicar OCP ayuda a limitar el efecto de los cambios, pero no evita todas las modificaciones.

12. Reflexiones

1. ¿Qué patrón resultó más natural y cuál fue más difícil de justificar?

El patrón que encontré más claro fue Strategy. La tarifa estándar y la exprés resuelven la misma tarea con fórmulas diferentes, así que separarlas en clases resulta útil. Builder fue más difícil de justificar porque Python ya permite usar argumentos nombrados. Lo conservé por la captura de datos en varios pasos y los campos opcionales; con una solicitud más corta usaría el constructor directamente.

2. ¿En qué se parecen y se diferencian Strategy y State?

Los dos patrones permiten delegar una tarea en objetos con una interfaz común. En EnvíoFácil, Strategy sirve para elegir cómo calcular el precio. State sería útil si el envío pasara de registrado a en tránsito y luego a entregado, y cada etapa permitiera acciones distintas. La diferencia está en el motivo del cambio: una política de cálculo en Strategy y una etapa del objeto en State. En este programa solo implementé Strategy.

3. ¿En qué escala trabajan eventos y Observer?

En este proyecto, Observer comunica objetos dentro del mismo programa: al registrar un envío, el publicador avisa al historial y a la consola. Una arquitectura orientada a eventos puede comunicar módulos o servicios separados. En ese caso habría que resolver también cómo guardar los mensajes, reintentarlos o evitar procesarlos dos veces. Mi implementación se limita a los avisos locales.

4. ¿Qué patrón eliminaría en un sistema mucho más pequeño?

Si el sistema fuera más pequeño, quitaría Builder. Con pocos campos sería más sencillo crear SolicitudEnvio directamente. Mantener un patrón sin una necesidad clara obliga a revisar más clases y hace más largo el recorrido por el código. También reconsideraría Factory Method si solo existiera un formato de guía.

5. ¿Qué conservaría al migrar de monolito a microservicios?

Conservaría las validaciones, las fórmulas de las tarifas, los creadores de guía y los contratos del componente. Cambiaría las clases que guardan información o llaman a otros sistemas. Si las notificaciones quedaran en un servicio separado, también tendría que ajustar su comunicación. La separación actual facilita esa tarea, aunque seguiría siendo necesario resolver errores de red, seguridad y coordinación entre servicios.

Conclusión del trabajo

La ventaja más clara del diseño es que el registro mantiene el mismo flujo aunque cambie la tarifa o la transportadora. Las 22 pruebas comprueban esas sustituciones y los principales casos de error. Para llevar el programa a una tienda real, el siguiente paso sería guardar la información de forma permanente y conectar los proveedores.

13. Alcance técnico y referencias

Decisiones y límites

Para el ejercicio definí una tarifa estándar de COP 7.000 más COP 2.000 por kilo y una exprés de COP 12.000 más COP 3.000 por kilo. Ambas añaden el 1 % del valor declarado. Son valores de ejemplo, no precios de una transportadora ni un seguro. La opción de paquete frágil agrega un aviso a la guía sin cambiar el costo.

Los registros duran mientras el programa está abierto. Esta versión permite estudiar el diseño y ejecutar los casos principales con almacenamiento en memoria, como admite la guía del laboratorio. Una aplicación de uso diario necesitaría persistencia, autenticación y conexión con servicios reales.

El programa solicita la guía antes de guardar el envío. Con una transportadora real habría que atender el caso en que la guía se genere, pero el guardado local falle. También harían falta reintentos para los avisos que no se logren enviar. Estos casos quedan fuera de la simulación.

La bitácora guarda el tipo de operación y el ID del envío cuando la operación termina bien. Las pruebas revisan resultados y errores esperados. La meta de tiempo de respuesta aparece como un criterio de diseño; no realicé una medición de rendimiento.

Referencias

[1] Vargas Pedraza, S. E. (s. f.). Laboratorio: del sector profesional al código. Arquitectura, sub-arquitectura, SOLID y patrones de diseño. Guía de Ingeniería de Software II (2016702), Universidad Nacional de Colombia, Sede Bogotá. Documento suministrado para esta actividad.

[2] Refactoring.Guru. (s. f.). El catálogo de patrones de diseño. Consulta conceptual de las familias y los seis patrones aplicados.
https://refactoring.guru/es/design-patterns/catalog

[3] Refactoring.Guru. (s. f.). Builder. Referencia para la construcción por pasos y sus costos.
https://refactoring.guru/design-patterns/builder

[4] Refactoring.Guru. (s. f.). Decorator. Referencia para la composición de objetos y la conservación de la interfaz.
https://refactoring.guru/design-patterns/decorator

[5] Python Software Foundation. (s. f.). unittest — Unit testing framework. Documentación oficial para descubrimiento y ejecución de pruebas.
https://docs.python.org/3/library/unittest.html

Los diagramas, las reglas ficticias del caso y los ejemplos de implementación corresponden al desarrollo de EnvíoFácil. Las referencias se usan como apoyo conceptual; no sustituyen la justificación de cada decisión del proyecto.